

Anforderungen an Synchronisations-Mechanismen:

Gegenseitiger Ausschluss (Nur ein Thread darf in Critical Section sein), Fortschritt (Entscheidung, wer in die Critical Section darf, muss zu endlich Zeit getroffen werden), Begrenztes Warten (Thread wird nur n mal übergeben, bevor er in die Critical Section darf).

Implementierung: Nur mit HW-Unterstützung möglich. Es gibt zwei atomare Instruktionen:

Test-And-Set (Setzt einen int auf 1 und returt den vorherigen Wert: `test_and_set(int * targeted) { int value = *target; *target = 1; return value; }`) und **Compare-and-Swap** (Überschreibt einen int auf einen spezifizierten Wert, wenn dieser dem erwarteten Wert entspricht: `compare_and_swap (int *a, int expected, int new) { int value = *a; (*value == expected) ? (*a = new) : return value; }`).

SEMAPHORE

Einzelni Zähler $z \geq 0$. Wird nur über `Post(v)` (Erhöhe z um 1) und `Wait(v)` zugegriffen (Wart: Wenn $z > 0$, verringere z um 1 und fahre fort, wenn $z = 0$, setz den Thread in den Wartezustand, bis anderer Thread z erhöht).

Alle diese Funktionen geben, gemäss POSIX API (Thread Library), den Status eines zürück, weggelürzt.

`sem_init (&sem_t, sem, int pshared, unsigned int value);`
Initialisiert Semaphore vom Typ `sem_t`. Sem_t ist eine globale Variable oder member von struct, pshared=0 auf einen Prozessor, value=Startwert von Semaphore.
Verwendung globaler Semaphore über mehrere Prozesse:

```
sem_t sem; /*global*/
int main (...) { sem_init (&sem, 0, 0); /* pshared=0, Semaphore Wert auf 0 */

sem_wait (&sem_t); //sem_wait (&sem, post (&sem_t, sem); //implementieren Post + Wait + sem_trywait (&sem_t, sem); sem_timedwait (&sem_t, sem, const struct timespec *abs_timeout); Sind wie sem_wait, aber brechen ab, falls Dekrement nicht durchgeführt werden kann, sem_trywait bricht sofort ab, sem_timedwait nach der angegebenen Zeitdauer.
sem_destroy (&sem_t); //sem_destroy (&sem_t, sem); Entfernt Speicher von Semaphore.

semaphore free = n; semaphore used = 0; // Beispiel mit 1 Producer und 1 Consumer
```

```
// Produces while (C) { if (C) WAIT (free); //warte auf Platz in Queue produce_item (Cwait[Platz], ...); POST (used); // 1 Element mehr in Queue free = (free+1) % BUFFER_SIZE; } // Consumes while (C) { if (C) WAIT (used); // Hat es Elem. in Queue? consume (Cbuffer[free]); POST (free); // 1 Element weniger in Q free = (free+1) % BUFFER_SIZE; } }
```

MUTEXE

Ein Mutex hat einen **binären Zustand** z , der nur durch zwei Funktionen verändert werden kann: **Acquire** (Wenn $z = 0$, setze z auf 1 und fahre fort, Wenn $z = 1$, blockiere den Thread, bis $z = 0$), **Release** (Setzt $z = 0$). Auch als non-blocking-Funktion: `int pthread_mutex_trylock (pthread_mutex_t *mutex)`

// Beispiel Initialisierung pthread_mutex_t mutex; // global int main() { pthread_mutex_init (&mutex, 0); // run threads & wait for them to finish pthread_mutex_lock (&mutex); // Leave critical section pthread_mutex_unlock (&mutex); ... }

Priority Inversion: Ein hoch-priorisierter Thread C wartet auf eine Ressource, die von einem niedriger priorisierten Thread A gehalten wird. Ein Thread mit Priorität zwischen diesen beiden Threads erhält den Prozessor. Kann mit **Priority Inheritance** gelöst werden. Die Priorität von A wird temporär auf die Priorität von C gesetzt, damit der Mutex schnell wieder freigegeben wird.

SIGNALS, PIPES UND SOCKETS

SIGNAL

Signale ermöglichen es, einen Prozess von **ausen** zu unterbrechen, wie ein **Interrupt**. Unterbrechen des gerade laufenden Prozesses/Threads, Auswahl und Ausführen der **Signal-Handler-Funktionen**, Fortsetzen des Prozesses. Werden über ungültige Instruktionen oder Abbruch auf Seitens Benutzer ausgelöst. Jeder Prozess hat pro Signal einen Handler.

Mit kill kann ein Signal, mit einem dieser Default-Handler, gesendet werden:
Abnormal-Termiante-Handler | **Beendet Programm** + **Core Dump** (für Programmfehler)
SIGSEGV (fehler in arithmetischen Operationen), **SIGILL** (linguistische Instruktion)
Termiante-Handler | **Beendet das Programm** (Prozess abbrechen)
SIGKILL (nicht überbrechbar), **SIGSTOP** (Normale lokale Anfrage an den Prozess, sich zu beenden), **SIGINT** (Nichtabrechbare Aufforderung an Prozess, sich zu beenden), **SIGQUIT** (Wie SIGINT, aber anomale Terminierung Ctrl-\), **SIGABRT** (Wie SIGQUIT, aber vom Prozess an sich selber (Programmierfehler bemerkli), **Ignore-Handler** | **Ignoriert das Signal**
SIGSTOP Versteht den Prozess in den Zustand **stoppen**, ähnlich wie **waiting Ctrl-Z**, **SIGSTOP** (Wie SIGSTOP, aber nicht überbrechbar), **SIGCONT** (Setzt den Prozess fort (fg))

`int sigaction(int signal, struct sigaction *new, struct sigaction *old);`
Setzt Signal-Handler für signal, wenn new != 0. (Signal-Signal-Handler definiert was signalaction struct: sa_handler; Zu callende Funktion, sa_mask: blockierte Signale während Ausführung, bearbeitet nur durch signalct() Funktion: signalpcttyp, sigfillset, sigdaddet, sigdelset, sigismember)
signalcn = transp. Datenstruktur, signal_t = opake Datenstn. = implementationsabhängig

PIPES

geöffnete Datei: entspricht einem **Eintrag** in der **File-Descriptor-Tabelle (FDT)** im Prozess. Zugriff über **File-API** (open, close, read, write,...). Das OS speichert je **Eintrag** der Prozess-FD ein **Verweis** auf die **globale FD**.
fork(): gibt eine **FD** weiter, die kopiert.

`int dup (int source_fd); int dup2 (int source_fd, int destination_fd);` Duplizieren und returen den FD source_fd. dup aliziert neuen FD, dup2 überschreibt destination_fd.

Umsleiten des Ausgabestreams
`int fd = open("log.txt", ...); int pid = fork(); if (pid == 0) { // child dup2(fd, 2); // duplicate fd for log.txt as standard output // e.g., load new image with execv, fd's remain // else { // parent <= close (fd); }`

(Eine Datei muss nur **write** & **close** & **read/write** unterstützen)
Pipe: «Daten im Hauptspeicher, die über zwei FD verwendet wird.
read end zum Lesen», **write** end zum Schreiben in die Pipe kein Lesek.

Daten, die in **write** end geschrieben werden, können aus **read** end genau einmal gemäss **FIFO** gelesen werden. Pipes erlauben Kommunikation über Prozess-Grenzen hinweg. In **unidirectional** int pipe (int fd [2]); // equivalent to int pipe (fd)

```
int fd[2]; // 0 = read, 1 = write if (fd == 0) { // Child thread pipe (fd [1]); // don't use write end char buffer [BSIZE]; int n = read (fd[0], buffer, BSIZE); // parent thread } else { // Parent thread pipe (fd[0]); // don't use read end char * text = "I <3 strawberries"; write (fd[1], text, strlen(text) + 1); }
```

unklar, wer die Daten erhält.
– keine Daten in der Pipe -> read blockiert, bis Daten hineingeschrieben werden
– keine Daten in der Pipe + kein geöffnetes Write-End -> read gibt 0 zurück (EOF)
– Default Pipe Größe = 16 Pages, meistens 64KB

`int mkfifo (const char *path, mode_t mode);` Erzeugt eine Pipe mit Namen und Pfad im Dateisystem. Hat via mode Permission Bits wie normale Datei. Lieh unabhängig vom erzeugenden Prozess, je nach System auch über Reboots hinweg. Muss explizit mit `unlink` gelöscht werden.

SOCKETS

Ein Socket repräsentiert einen **Endpunkt** auf einer Maschine. Kommunikation findet im Regelfall zwischen zwei Sockets statt (JUDG TCP über IP sowie Unix Domain Sockets).

Sockets benötigen für Kommunikation einen Namen: (IP-Adresse & Portnummer)
`int socket(int domain, int type, int protocol);` Erzeugt einen neuen Socket als «Daten». Socket wird nach Erzeugung **zuerst unbenannt**. Alle Operationen blockieren per default.
Domain (AF_INET, AF_INET6, type (SOCK_STREAM, SOCK_DGRAM, SOCK_STREAM, protocol (System-specific, 0 = Default-Protocol)
Client: connect (Verbindung unter Angabe einer Adresse aufbauen), listen (Senden von Daten, 0 => no call), read (Empfangen von Daten, 0 => no call), close (Schliessen der Verbindung)

Server: bind (festlegen einer nach aussen sichtbaren Adresse), listen (Bereitstellen einer Queue zum Sammeln von Verbindungsanfragen von Clients), accept (Erzeugen einer Verbindung auf Anfrage von Client), read (Empfangen von Daten, 0 => no call), write (Senden von Daten, 0 => no call), close (Schliessen der Verbindung)

```
struct sockaddr_in ip_addr;
ip_addr.sin_port = htons (443); // default HTTPS port
inetpton (AF_INET, "192.168.0.1", &ip_addr.sin_addr.s_addr); // IP address in memory: 0x00 0x80 0x00 0x01, Port in memory: 0x01 0xb8
htons() konvertiert 16 Bit von Host-Byte-order (x) zu Network-Byte-Order (y), htons() 32 Bit.
ntohs() und ntohs() sind Gegenstücke. inetpton() konvertiert protokol-spezifische Adresse von String zu Network-8. inet_ntop() ist das Gegenstück (network-to-presentation).
int bind (int socket, const struct sockaddr *local_address, socklen_t, addrlen); bind bindet den Socket an die angegebene, unbentzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis der Vorgang abgeschlossen ist.
int connect (int socket, struct sockaddr *remote_address, socklen_t, addrlen); Aufbau einer Verbindung. bindet den Socket an eine neue, unbentzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis Verbindung steht oder ein Timeout eintritt.
int listen (int socket, int backlog); Markiert den Socket als «bereit zum Empfang von Verbindungen». Erzeugt eine Warteschlange, die so viele Verbindungsanfragen aufnehmen kann, wie backlog angibt.
int accept (int socket, struct sockaddr *remote_address, socklen_t, address_len);
// Wartet, bis Verbindungsanfrage in der Warteschlange eintrifft. Erzeugt einen neuen Socket und bindet ihn an eine neue lokale Adresse. Die Adresse des Clients wird in remote_address geschrieben. Der neue Socket kann keine weiteren Verbindungen annehmen, der bestehende schon.
```

Typisches Muster für Server
`int server_fd = socket (...); bind (server_fd, ...); listen (server_fd, ...); while (connectfd = accept (server_fd, 0, 0); delegate_to_worker_thread (clientfd); // will call close(client_fd)`

```
ssize_t send (int socket, const void *buffer, size_t length, int flags); // wie write: * ssize_t write (int fd, void *buffer, size_t n)
ssize_t recv (int socket, void *buffer, size_t length, int flags); // wie read: * ssize_t read (int fd, void *buffer, size_t n)
send (fd, buf, len, 0); // Aufruf Beispiel
```

Senden und Empfangen von Daten
Erweitern read() bzw. write() um Socket-Funktionalitäten durch den flags-Parameter. Puffern der Daten ist Aufgabe des Netzwerkstacks.

`int close (int socket);` Schliesst Socket für den aufrufenden Prozess. Hat anderer Prozess den Socket geöffnet, bleibt Verbindung bestehen. Gegenseite wird nicht benachrichtigt. Gegenseite erhält bei einem read kein EOF und verlässt die Lese-Schleife nicht.
`int shutdown (int socket, int mode);` Schliesst Socket für alle Prozesse und baut die entsprechende Verbindung ab. mode: SHUT_RD (keine Lese-Zugriffe mehr), SHUT_WR (keine Schreib-Zugriffe mehr), SHUT_RDWR (keine Lese- oder Schreib-Zugriffe mehr)

MESSAGE PASSING UND SHARED MEMORY AND PIPES
Prozesse sind voneinander isoliert, müssen jedoch trotzdem miteinander interagieren.

Shared Memory ist schneller zu realisieren, aber schwer wartbar. Message-Passing erfordert mehr Engineering-Aufwand, schlussendlich aber in Mehr-Prozessor-Systemen bald performanter.

MESSAGE-PASSING
ist ein Mechanismus mit zwei Operationen:
– **Send** (kopiert die Nachricht aus dem Prozess: send (message)),
– **Receive** (kopiert die Nachricht in den Prozess: receive (message)).

Unterscheidungskriterien
Dabei können Implementierungen nach verschiedenen Kriterien unterschieden werden

Feste oder variable Nachrichtengrösse: feste Nachrichtengrösse ist einfacher zu implementieren, aber umständlicher zu verwenden als variable Nachrichtengrösse.
Direkte Kommunikation: Kommunikation nur zwischen genau zwei Prozessen, Sender muss Empfänger kennen. Kann es sich symmetrische direkte Kommunikation (Empfänger muss Sender kennen, können jedoch die ID in einem Ctl-Parameter mit liefern (recv_id, message)).
Indirekte Kommunikation: Prozess sendet Nachricht an Mailboxen, Ports oder Queues. Empfänger empfängt aus diesem Objekt. Beide Teilnehmer müssen die gleichen() Mailbox(en) kennen.
Weitere: synchrone oder asynchrone Kommunikation, Pufferung, mit oder ohne Prioritäten für Nachrichten

Indirekte Kommunikation
Lebenszyklus-Queue: Wenn diese Queue einen Prozess herkö, lebt sie solange wie der Prozess. Wenn sie dem OS gehört, muss das OS das Löschen übernehmen.

Synchronisation: Blockierendes Sendendes (Sender wird solange blockiert, bis die Nachricht vom Empfänger empfangen wurde), Nicht-blockierendes Sendendes (Sender sendet Nachricht und fährt sofort weiter), Blockierendes empfangen (Empfänger wird blockiert, bis Nachricht verfügbar), Nicht-blockierendes empfangen (Empfänger erhält Nachricht, wenn verfügbar, aber nicht).

```
// Producer message msg; open(O); while(1) { produce_next(msg); send(O, msg); // blocked until sent } // Consumer message msg; open(O); while(1) { receive(O, msg); // blocked until rec. consume_next(msg); }
```

Pufferung: **Keine** (Queue-Länge ist 0, Sender muss blockieren), **Beschränkte** (Maximal n Nachrichten, Sender blockiert, wenn Queue voll ist), **Unbeschränkte** (Beliebig viele Nachrichten, Sender blockiert nie).

Prioritäten: In manchen Systemen können Nachrichten mit **Prioritäten** versehen werden. Der Empfänger holt die Nachricht mit der **höchsten Priorität** zuerst aus der Queue.

POSIX Message-Passing = OS Message Queue API

Nutzt OS Message-Queues mit **variabler Länge**, haben mind. 32 Prioritäten und können **synchron** und **asynchron** verwendet werden, indirekte Kommunikation.
`msg_t m = msg_create (const char *name, int flags, mode, struct msg_attr *attr);` Öffnet eine Message-Queue mit systemtemem name, returnt Message-Queue-Descriptor. (name mit *w/ beginnt, flags & mode wie bei Dateien, msg_attr: Dtl. Konfig & Queue-Status, R/W mit w/getattr/setattr)
`int mq_close (mqd_t mq);` Schliesst die Queue mit dem Descriptor queue für diesen Prozess.
`int mq_unlink (const char *name);` Entfernt die Queue mit dem Namen name aus dem System. Name wird sofort entfernt und existiert nicht mehr geöffnet werden.
`int mq_send (mqd_t mq, const char *msg, size_t length, unsigned int priority);` Sendet die Nachricht, die an Adresse msg beginnt und length Bytes lang ist, in die queue.
`int mq_receive (mqd_t mq, const char *msg, size_t length, unsigned int *priority);` Kopiert die nächste Nachricht aus der Queue in den Puffer, der an Adresse msg beginnt und length Bytes lang ist. Blockiert, wenn die Queue leer ist.

Message-Queues	Pipes
– bidirektional	– unidirektional
– Daten sind in einzelnen Messages organisiert	– übermitteltes Bytestrom an Daten
– beliebiger Zugriff	– beliebiger Zugriff
– Haben immer einen Namen	– Müssen keinen Namen haben

SHARED MEMORY

Frames des Hauptspeichers werden **zwei (oder mehr) Prozessen P₁ und P₂** zugänglich gemacht. In P_1 wird Page V_1 auf einen Frame F abgebildet. In P_2 wird Page V_2 auf denselben Frame F abgebildet. Beide Prozesse können beliebig auf dieselben Daten zugreifen. Im Shared Memory müssen **relative Adressen** verwendet werden.

POSIX API

Das OS benötigt eine «Datei S », das Informationen über den gemeinsamen Speicher verwaltet und eine **Mapping Table** je Prozess.

POSIX (mode für shm, open: s_IRWXU = 0700, S_IRUSR = 0200, S_IWUSR = 0040, S_IRGRP = 0001, S_IWGRP = 1) verknüpft.
`shm_open (C"/varshare/daemon", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);` Erzeugt (falls nicht) einen offener Shared Memory /varshare/daemon zum Lesen und Schreiben.
`int ftruncate (int fd, off_t length);` Setzt Grösse der «Datei». Muss zwingend nach SM-Erstellung gesetzt werden, um entsprechend viele Frames zu allozieren.
Für Shared Memory mit ganzzahligen Vielfachen der Frame-/Framegrösse verwendet.
`int close (int fd);` Schliesst «Datei». Shared Memory bleibt aber im System.
`int shm_unlink (const char * name);` Löscht das Shared Memory mit dem name. (bleibt vorhanden, solange noch von Prozess geöffnet ist)
`int shmmap (void *address, size_t length);` Entfernt das Mapping.
`void * address = mmap (0, maps shared memory into virt. address space of process 0); // void init_address (0 because nobody cares)
size_of_shared_memory; // size_t length (same as used in ftruncate)
PROT_READ | PROT_WRITE, // int protection (never use execute)
MAP_SHARED, // int flags
fd, // int file descriptor,
0; // off_t offset (start map from first byte)`

UNICODE

ASCII - AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE
Hex 10101A, 1011B, 1100C, 1101D, 1110E, 1111F

Hat 128 mögliche Zeichen (erste Hexzahl = Zeile x , zweite Hexzahl = Spalte y , d.h. ax, by).

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	MIL	DEL	1	2	3	4	5	6	7	8	9	A	B	C	D	E
1																
2	U	I	"	"	"	"	"	"	"	"	"	"	"	"	"	"
3	0	1	2	3	4	5	6	7	8	9	<	>	<	>	<	>
4	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	]	^	_	`
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	}	~		

Codepages: unabhängige Erweiterungen auf 8 Bit. Jede ist unterschiedlich und nicht erkennbar. **Unicode:** Hat zum Ziel, einen eindeutigen Code für **jedes vorhandene Zeichen** zu definieren. D8,00, bis DF FF, sind wegen UTF-16 keine gültigen Code-Points.

Code-Points (CP oder P#): Nummer eines Zeichens - «welches Zeichen?» in der Unicode Tabelle
Code-Unit (CU): Einheit, um Zeichen in einem Encoding darzustellen (bietet den Speicherplatz)
 $P_1 = 1$ = 1es Bit des uncodierten CPs, $U_1 = 1$ = 1es Code-Unit des kodierten CPs, $D_1 = 1$ = 1es Byte des kodierten CPs

Jede CU ist die ganze 32 Bit gross, jeder CP ist eine CU. Wird häufig intern in Programmen verwendet. Obere 11 Bits oft «zweckentfremdet». Endanfang gibt an, mit welchem Ende die Folge anfangt.

Big-Endian: 83 B2 B1 B0 0 aus Unicode-Tabelle,
Little-Endian: 80 B1 B2 B3

UTF-8

Jede CU umfasst 8 Bit, ein CP benötigt 1 bis 4 CUs. Standard für Webpages. Echte Erweiterung von ASCII. Jedes B (Byte) = 1 (Code-Unit), Maximal 4 Bytes.

Code-Point in	U_0	U_1	U_2	U_3	signifikant
0...7F _h					7 bits
80...7F _h					11 bits
800...FF _h					16 bits
10000...10FFF _h	11110xxx _h	10xxx xxx _h	10xxx xxx _h	10xxx xxx _h	21 bits

Vorgehen UTF-8

- $P = E_4 =$
- in Binär umwandeln 1110 0100
- von rechter Seite her die Bits aufteilen und Separator-Bits korrekt einfügen:
1 | 11010010 -> 110 01001 10 100100
3. am Schluss zurück in Hex umwandeln: C3 44

UTF-16

Jede CU umfasst 16 Bit, ein CP benötigt 1 oder 2 CUs. Die 2 CUs werden **Surrogate Pair** genannt, U_0 : high surrogate, U_1 : low surrogate. Bei 2 Bytes (1 cu) wird Kombination mit vorne mit Nullen angereiht, CP in 0 - 1FFFF_h (D800 - DFFF_h). Bei 4 Bytes sind D8 00, bis DF FF_h (Bis 12722) wegen dem Separator **ungültig** und müssen «umgerechnet» werden.

BE: U0 = 81 B0, LE: U0 = 80 B1
BE: U1 U0 = 83 B2 81 B0, LE: U1 U0 = 82 83 80 B1
-1'0000, 0...1'0000, B₀ B₁ B₂ B₃ B₄ B₅ B₆ B₇ B₈ B₉ B₁₀ B₁₁ B₁₂ B₁₃ B₁₄ B₁₅ B₁₆ B₁₇ B₁₈ B₁₉ B₂₀ B₂₁ B₂₂ B₂₃ B₂₄ B₂₅ B₂₆ B₂₇ B₂₈ B₂₉ B₃₀ B₃₁ B₃₂ B₃₃ B₃₄ B₃₅ B₃₆ B₃₇ B₃₈ B₃₉ B₄₀ B₄₁ B₄₂ B₄₃ B₄₄ B₄₅ B₄₆ B₄₇ B₄₈ B₄₉ B₅₀ B₅₁ B₅₂ B₅₃ B₅₄ B₅₅ B₅₆ B₅₇ B₅₈ B₅₉ B₆₀ B₆₁ B₆₂ B₆₃ B₆₄ B₆₅ B₆₆ B₆₇ B₆₈ B₆₉ B₇₀ B₇₁ B₇₂ B₇₃ B₇₄ B₇₅ B₇₆ B₇₇ B₇₈ B₇₉ B₈₀ B₈₁ B₈₂ B₈₃ B₈₄ B₈₅ B₈₆ B₈₇ B₈₈ B₈₉ B₉₀ B₉₁ B₉₂ B₉₃ B₉₄ B₉₅ B₉₆ B₉₇ B₉₈ B₉₉ B₁₀₀ B₁₀₁ B₁₀₂ B₁₀₃ B₁₀₄ B₁₀₅ B₁₀₆ B₁₀₇ B₁₀₈ B₁₀₉ B₁₁₀ B₁₁₁ B₁₁₂ B₁₁₃ B₁₁₄ B₁₁₅ B₁₁₆ B₁₁₇ B₁₁₈ B₁₁₉ B₁₂₀ B₁₂₁ B₁₂₂ B₁₂₃ B₁₂₄ B₁₂₅ B₁₂₆ B₁₂₇ B₁₂₈ B₁₂₉ B₁₃₀ B₁₃₁ B₁₃₂ B₁₃₃ B₁₃₄ B₁₃₅ B₁₃₆ B₁₃₇ B₁₃₈ B₁₃₉ B₁₄₀ B₁₄₁ B₁₄₂ B₁₄₃ B₁₄₄ B₁₄₅ B₁₄₆ B₁₄₇ B₁₄₈ B₁₄₉ B₁₅₀ B₁₅₁ B₁₅₂ B₁₅₃ B₁₅₄ B₁₅₅ B₁₅₆ B₁₅₇ B₁₅₈ B₁₅₉ B₁₆₀ B₁₆₁ B₁₆₂ B₁₆₃ B₁₆₄ B₁₆₅ B₁₆₆ B₁₆₇ B₁₆₈ B₁₆₉ B₁₇₀ B₁₇₁ B₁₇₂ B₁₇₃ B₁₇₄ B₁₇₅ B₁₇₆ B₁₇₇ B₁₇₈ B₁₇₉ B₁₈₀ B₁₈₁ B₁₈₂ B₁₈₃ B₁₈₄ B₁₈₅ B₁₈₆ B₁₈₇ B₁₈₈ B₁₈₉ B₁₉₀ B₁₉₁ B₁₉₂ B₁₉₃ B₁₉₄ B₁₉₅ B₁₉₆ B₁₉₇ B₁₉₈ B₁₉₉ B₂₀₀ B₂₀₁ B₂₀₂ B₂₀₃ B₂₀₄ B₂₀₅ B₂₀₆ B₂₀₇ B₂₀₈ B₂₀₉ B₂₁₀ B₂₁₁ B₂₁₂ B₂₁₃ B₂₁₄ B₂₁₅ B₂₁₆ B₂₁₇ B₂₁₈ B₂₁₉ B₂₂₀ B₂₂₁ B₂₂₂ B₂₂₃ B₂₂₄ B₂₂₅ B₂₂₆ B₂₂₇ B₂₂₈ B₂₂₉ B₂₃₀ B₂₃₁ B₂₃₂ B₂₃₃ B₂₃₄ B₂₃₅ B₂₃₆ B₂₃₇ B₂₃₈ B₂₃₉ B₂₄₀ B₂₄₁ B₂₄₂ B₂₄₃ B₂₄₄ B₂₄₅ B₂₄₆ B₂₄₇ B₂₄₈ B₂₄₉ B₂₅₀ B₂₅₁ B₂₅₂ B₂₅₃ B₂₅₄ B₂₅₅ B₂₅₆ B₂₅₇ B₂₅₈ B₂₅₉ B₂₆₀ B₂₆₁ B₂₆₂ B₂₆₃ B₂₆₄ B₂₆₅ B₂₆₆ B₂₆₇ B₂₆₈ B₂₆₉ B₂₇₀ B₂₇₁ B₂₇₂ B₂₇₃ B₂₇₄ B₂₇₅ B₂₇₆ B₂₇₇ B₂₇₈ B₂₇₉ B₂₈₀ B₂₈₁ B₂₈₂ B₂₈₃ B₂₈₄ B₂₈₅ B₂₈₆ B₂₈₇ B₂₈₈ B₂₈₉ B₂₉₀ B₂₉₁ B₂₉₂ B₂₉₃ B₂₉₄ B₂₉₅ B₂₉₆ B₂₉₇ B₂₉₈ B₂₉₉ B₃₀₀ B₃₀₁ B₃₀₂ B₃₀₃ B₃₀₄ B₃₀₅ B₃₀₆ B₃₀₇ B₃₀₈ B₃₀₉ B₃₁₀ B₃₁₁ B₃₁₂ B₃₁₃ B₃₁₄ B₃₁₅ B₃₁₆ B₃₁₇ B₃₁₈ B₃₁₉ B₃₂₀ B₃₂₁ B₃₂₂